



---

# MAZE GENERATOR & SOLVER SYSTEM

---

Technical Documentation



MAY 13, 2026

ADDIS ABABA UNIVERSITY  
Addis Ababa Ethiopia

## Contents

|                                                                              |    |
|------------------------------------------------------------------------------|----|
| Maze Generator & Solver Visualization System.....                            | 5  |
| Technical Documentation.....                                                 | 5  |
| 1. Project Overview.....                                                     | 5  |
| Project Title.....                                                           | 5  |
| Interactive Maze Generator and Solver Visualization .....                    | 5  |
| Project Description.....                                                     | 5  |
| 2. Assignment Objectives.....                                                | 6  |
| Generate a Maze .....                                                        | 6  |
| Solve the Maze.....                                                          | 6  |
| Demonstrate Algorithms.....                                                  | 6  |
| Provide Visualization .....                                                  | 6  |
| 3. Functional Requirements.....                                              | 6  |
| 3.1 Maze Generation.....                                                     | 6  |
| 3.2 Maze Solving.....                                                        | 7  |
| 3.3 Visualization.....                                                       | 7  |
| 3.4 User Interaction.....                                                    | 7  |
| 4. Non-Functional Requirements.....                                          | 8  |
| 5. Technology Stack.....                                                     | 8  |
| 6. Why Babylon.js.....                                                       | 9  |
| 7. System Architecture Architectural Style Modular Layered Architecture..... | 9  |
| 8. High-Level System Design .....                                            | 10 |
| 9. Project Structure .....                                                   | 10 |
| 10. Core Data Structures .....                                               | 11 |
| 10.1 Cell Representation.....                                                | 11 |
| 10.2 Maze Representation.....                                                | 12 |
| 10.3 Stack Structure.....                                                    | 12 |
| 11. Maze Generation Algorithm .....                                          | 12 |

|                                    |    |
|------------------------------------|----|
| Algorithm Type.....                | 12 |
| Randomized Depth-First Search..... | 13 |
| Steps .....                        | 13 |
| Generation Pseudocode .....        | 14 |
| 12. Maze Solving Algorithm .....   | 14 |
| Algorithm Type.....                | 14 |
| DFS Backtracking Solver.....       | 14 |
| Solver Steps.....                  | 14 |
| Solver Pseudocode.....             | 15 |
| 13. Rendering System.....          | 16 |
| Rendering Components .....         | 16 |
| 14. Graphics Pipeline .....        | 16 |
| 15. Animation System.....          | 16 |
| Animation States.....              | 16 |
| 16. Camera System.....             | 17 |
| Primary Camera .....               | 17 |
| Optional Cameras.....              | 17 |
| 17. Material & Color System.....   | 17 |
| 18. Bonus Features.....            | 18 |
| Cyclic Mazes.....                  | 18 |
| Additional Features.....           | 18 |
| 19. Complexity Analysis .....      | 18 |
| Maze Generation.....               | 18 |
| Maze Solving.....                  | 19 |
| 20. Development Roadmap.....       | 19 |
| 21. Testing Strategy.....          | 21 |
| 22. GitHub Strategy.....           | 22 |
| 23. README Structure.....          | 22 |
| 24. Risk Analysis .....            | 22 |
| 25. Final Expected Output.....     | 23 |

|                                                     |    |
|-----------------------------------------------------|----|
| 26. Conclusion .....                                | 23 |
| 1.Maze Generator & Solver Visualization System..... | 24 |
| System Design Document.....                         | 24 |
| 1. System Overview .....                            | 25 |
| 2. High-Level Architecture .....                    | 25 |
| 3. Architectural Pattern.....                       | 26 |
| 4. Core Subsystems.....                             | 26 |
| 5. Core Engine Design.....                          | 27 |
| 6. Maze Data Model Design .....                     | 28 |
| Maze Representation .....                           | 28 |
| 7. Maze Generator System Design.....                | 29 |
| 8. Solver System Design.....                        | 30 |
| 9. Rendering System Design.....                     | 31 |
| 10. Mesh Design.....                                | 32 |
| 11. Animation System Design.....                    | 33 |
| 12. Event System Design .....                       | 33 |
| 13. State Management Design.....                    | 34 |
| 14. Camera System Design.....                       | 34 |
| 15. Material System Design .....                    | 35 |
| 16. UI System Design .....                          | 36 |
| 17. Collision Design .....                          | 36 |
| 18. Memory Design.....                              | 37 |
| 19. Error Handling Design .....                     | 37 |
| 20. Scalability Design .....                        | 38 |
| 21. Future Extensibility.....                       | 38 |
| 22. UML-Style Component Relationships .....         | 38 |
| 23. Rendering Lifecycle .....                       | 39 |
| 24. Deployment Design.....                          | 39 |
| 25. Security Considerations.....                    | 40 |
| 26. Final System Behavior .....                     | 40 |

27. Design Summary..... 41

# Maze Generator & Solver Visualization System

Technical Documentation

---

## 1. Project Overview

Project Title

## Interactive Maze Generator and Solver Visualization

---

### Project Description

This project is a real-time interactive maze generation and maze solving visualization system built using modern web-based computer graphics technologies.

The system dynamically:

- generates a perfect maze using DFS-based recursive backtracking,
- visualizes the wall-carving process,
- solves the maze using stack-based backtracking,
- displays traversal and dead-end discovery,
- and renders the entire process in a 3D interactive environment.

The project is designed for a Computer Graphics assignment focused on:

- procedural generation,
  - graph traversal algorithms,
  - animation,
  - visualization,
  - and interactive rendering.
-

## 2. Assignment Objectives

The project must:

### Generate a Maze

- rectangular grid maze,
- random generation,
- proper/perfect maze,
- dynamically visualized generation.

### Solve the Maze

- automatically traverse maze,
- show exploration visually,
- backtrack at dead ends,
- highlight solution path.

### Demonstrate Algorithms

- stack-based DFS,
- backtracking,
- randomized traversal.

### Provide Visualization

- real-time rendering,
  - wall removal animation,
  - traversal animation,
  - dead-end highlighting.
- 

## 3. Functional Requirements

### 3.1 Maze Generation

The system shall:

- generate an  $R \times C$  maze,

- initialize all walls,
  - randomly carve paths,
  - ensure every cell is reachable,
  - ensure only one path exists between any two cells.
- 

## 3.2 Maze Solving

The system shall:

- find path from start to goal,
  - visualize movement,
  - identify dead ends,
  - backtrack dynamically,
  - display final successful path.
- 

## 3.3 Visualization

The system shall:

- render maze walls,
  - animate wall destruction,
  - render mouse traversal,
  - show visited cells,
  - show dead-end cells,
  - highlight solution path.
- 

## 3.4 User Interaction

The user shall be able to:

- generate new maze,
- restart solver,
- control animation speed,
- switch camera view,
- optionally toggle bonus features.

---

## 4. Non-Functional Requirements

### Performance

- smooth rendering at 60 FPS,
- support medium-to-large mazes,
- efficient memory usage.

### Maintainability

- modular architecture,
- clean separation of logic/rendering,
- reusable components.

### Scalability

- support future algorithms,
- support additional rendering modes,
- support larger environments.

### Readability

- documented code,
- meaningful naming,
- structured folders.

---

## 5. Technology Stack

| Layer           | Technology               |
|-----------------|--------------------------|
| Language        | TypeScript               |
| Graphics Engine | Babylon.js               |
| Build Tool      | Vite                     |
| Rendering       | WebGL/WebGPU via Babylon |
| Styling         | CSS                      |
| Version Control | Git + GitHub             |
| Recording       | Loom                     |

---

## 6. Why Babylon.js

Babylon.js was selected because it provides:

- high-level 3D abstractions,
- real-time rendering,
- mesh management,
- cameras,
- lighting,
- animation systems,
- debugging tools,
- efficient scene graph architecture.

Advantages:

- easier than raw WebGL,
- faster development,
- ideal for procedural visualization,
- excellent for educational graphics projects.

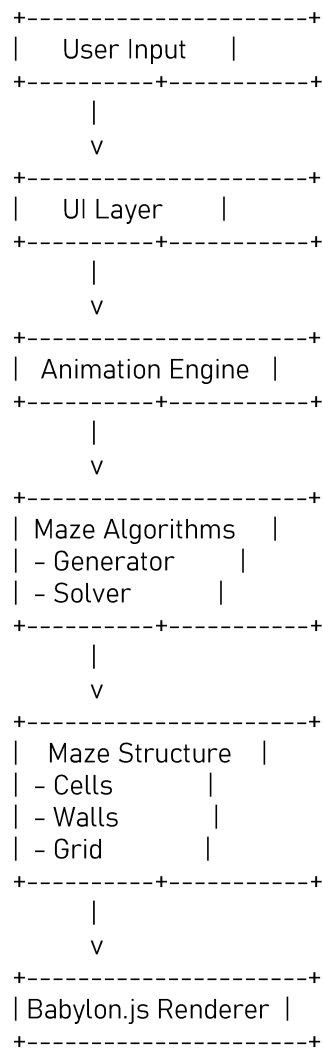
---

## 7. System Architecture Architectural Style Modular Layered Architecture

The project is divided into:

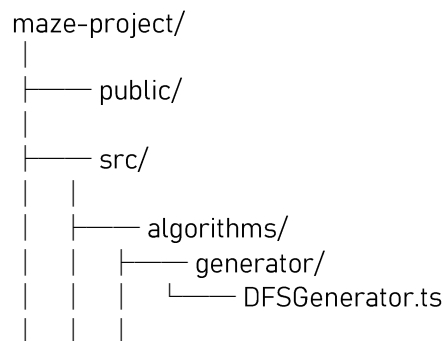
1. Core engine layer
  2. Maze logic layer
  3. Algorithm layer
  4. Rendering layer
  5. Animation layer
  6. UI layer
-

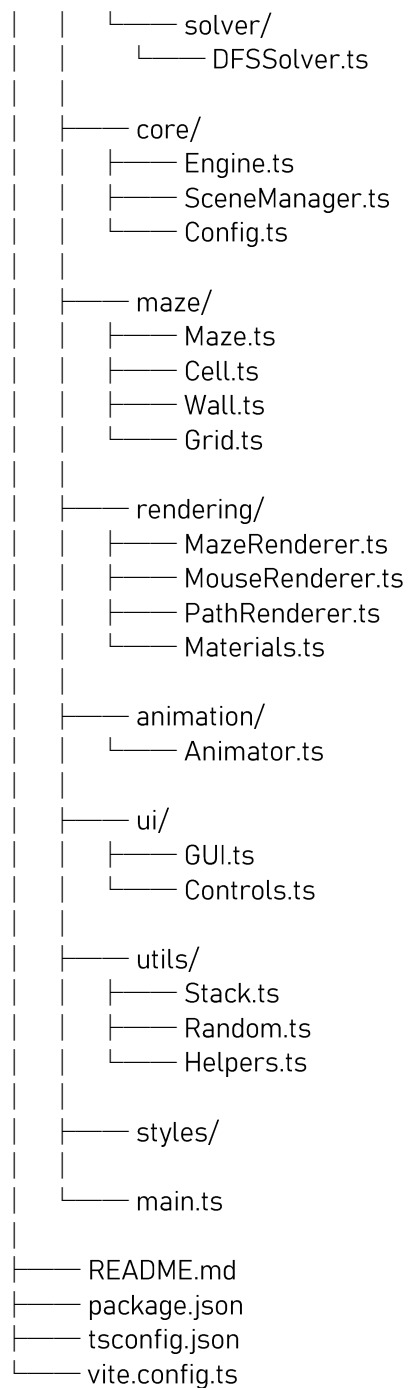
## 8. High-Level System Design



---

## 9. Project Structure





---

## 10. Core Data Structures

### 10.1 Cell Representation

Each maze cell contains:

```
interface Cell {  
    row: number;  
    col: number;  
  
    visited: boolean;  
  
    northWall: boolean;  
    eastWall: boolean;  
    southWall: boolean;  
    westWall: boolean;  
  
    isDeadEnd: boolean;  
    isSolutionPath: boolean;  
}
```

---

## 10.2 Maze Representation

Cell[]

The maze grid is a 2D matrix.

---

## 10.3 Stack Structure

Used in:

- generation,
- solving,
- backtracking.

```
class Stack<T> {  
    push(item: T): void  
    pop(): T | undefined  
    peek(): T | undefined  
    isEmpty(): boolean  
}
```

---

# 11. Maze Generation Algorithm

## Algorithm Type

# Randomized Depth-First Search

---

## Steps

Step 1

Initialize all walls intact.

Step 2

Choose random starting cell.

Step 3

Mark current cell visited.

Step 4

Find unvisited neighbors.

Step 5

Randomly select neighbor.

Step 6

Remove wall between cells.

Step 7

Push current cell onto stack.

Step 8

Move to selected neighbor.

Step 9

If dead end:

- pop stack,
- backtrack.

Step 10

Continue until stack empty.

---

## Generation Pseudocode

```
push(start)

while stack not empty:
    current = stack.peek()

    neighbors = unvisitedNeighbors(current)

    if neighbors exist:
        next = random(neighbors)

        removeWall(current, next)

        mark next visited

        push(next)

    else:
        pop()
```

---

## 12. Maze Solving Algorithm

### Algorithm Type

### DFS Backtracking Solver

---

## Solver Steps

Step 1

Start at maze entrance.

## Step 2

Try valid directions randomly.

## Step 3

Move if no wall exists.

## Step 4

Push path onto stack.

## Step 5

If trapped:

- mark dead end,
- backtrack.

## Step 6

Repeat until exit found.

---

# Solver Pseudocode

```
push(start)
```

```
while stack not empty:  
    current = stack.peek()
```

```
    if current == goal:  
        success
```

```
    next = availableNeighbor()
```

```
    if next exists:  
        push(next)
```

```
    else:  
        mark dead end  
        pop()
```

---

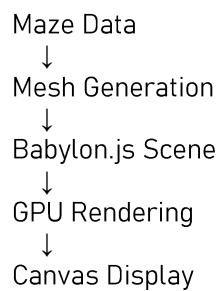
## 13. Rendering System

### Rendering Components

| Component           | Purpose                 |
|---------------------|-------------------------|
| Floor Mesh          | Maze ground             |
| Wall Meshes         | Maze structure          |
| Mouse Mesh          | Traversal visualization |
| Path Highlights     | Solution visualization  |
| Dead-End Indicators | Backtracking display    |

---

## 14. Graphics Pipeline



## 15. Animation System

The animation engine controls:

- generation timing,
  - solver movement,
  - wall destruction effects,
  - path updates.
- 

### Animation States

| State      | Description      |
|------------|------------------|
| Idle       | Waiting          |
| Generating | Building maze    |
| Solving    | Traversing maze  |
| Completed  | Final path shown |

---

## 16. Camera System

### Primary Camera

ArcRotateCamera

Advantages:

- easy navigation,
  - zoom support,
  - top-down visibility,
  - ideal for debugging.
- 

### Optional Cameras

| Camera          | Purpose        |
|-----------------|----------------|
| UniversalCamera | First-person   |
| FreeCamera      | Exploration    |
| FollowCamera    | Mouse tracking |

---

## 17. Material & Color System

| Object        | Color      |
|---------------|------------|
| Walls         | Gray/White |
| Floor         | Dark       |
| Current Mouse | Red        |
| Dead Ends     | Blue       |

|            |       |
|------------|-------|
| Object     | Color |
| Final Path | Green |

---

## 18. Bonus Features

### Cyclic Mazes

Randomly remove extra walls:

1 in 20 probability

Creates loops/cycles.

---

### Additional Features

Possible enhancements:

- FPS mode,
  - minimap,
  - statistics,
  - multiple algorithms,
  - lighting effects,
  - shadow system,
  - sound effects.
- 

## 19. Complexity Analysis

### Maze Generation

#### Time Complexity

$$O(R \times C)$$

Each cell visited once.

---

## Space Complexity

$$O(R \times C)$$

Due to stack + grid storage.

---

## Maze Solving

### Time Complexity

$$O(R \times C)$$

Worst case explores entire maze.

---

## 20. Development Roadmap

### Phase 1 — Project Setup

Duration: 1 Day

Tasks:

- initialize Vite,
- install Babylon.js,
- setup TypeScript,
- configure GitHub.

---

## Phase 2 — Maze Logic

Duration: 1–2 Days

Tasks:

- cell system,
- wall system,
- stack implementation,
- DFS generation.

---

## Phase 3 — Rendering

Duration: 1–2 Days

Tasks:

- render grid,
- render walls,
- materials,
- camera setup.

---

## Phase 4 — Animation

Duration: 1 Day

Tasks:

- generation animation,
- traversal animation,
- timing control.

---

## Phase 5 — Solver

Duration: 1 Day

Tasks:

- DFS solving,
- backtracking visualization,
- dead-end coloring.

---

## Phase 6 — Polish

Duration: 1 Day

Tasks:

- optimization,
- UI,
- README,
- recording.

---

# 21. Testing Strategy

## Unit Tests

Test:

- stack behavior,
- wall removal,
- neighbor detection,
- traversal validity.

---

## Integration Tests

Test:

- generation correctness,
- maze solvability,
- rendering synchronization.

---

## Visual Testing

Verify:

- animation smoothness,
  - correct coloring,
  - proper wall placement.
- 

## 22. GitHub Strategy

Recommended commits:

Initial Babylon.js setup  
Implemented maze cell structure  
Added DFS maze generation  
Implemented wall rendering  
Added maze generation animation  
Implemented DFS solver  
Added dead-end visualization  
Added final path highlighting  
Optimized rendering  
Completed documentation

---

## 23. README Structure

Recommended sections:

1. Project Overview
  2. Features
  3. Technologies
  4. Algorithms
  5. Installation
  6. Usage
  7. Screenshots
  8. Loom Demo
  9. Folder Structure
  10. Future Improvements
- 

## 24. Risk Analysis

| Risk             | Mitigation           |
|------------------|----------------------|
| Rendering lag    | mesh optimization    |
| Algorithm bugs   | unit testing         |
| Animation desync | centralized animator |

## 25. Final Expected Output

The final system should:

- dynamically generate mazes,
  - animate wall carving,
  - visually solve the maze,
  - show dead ends and backtracking,
  - provide smooth interactive graphics,
  - and maintain clean modular architecture.
- 

## 26. Conclusion

This project combines:

- computer graphics,
- procedural generation,
- data structures,
- graph algorithms,
- animation,
- and interactive visualization.

The chosen architecture using:

- TypeScript,
- Babylon.js,
- and modular DFS systems

provides:

- scalability,
- maintainability,
- professional structure,
- and strong visualization capability suitable for both academic evaluation and portfolio presentation.

# 1.Maze Generator & Solver Visualization System

## System Design Document

---

# 1. System Overview

The system is an interactive real-time visualization platform that:

- procedurally generates mazes,
- visualizes maze construction,
- solves the maze automatically,
- animates traversal and backtracking,
- and renders the process in a 3D environment.

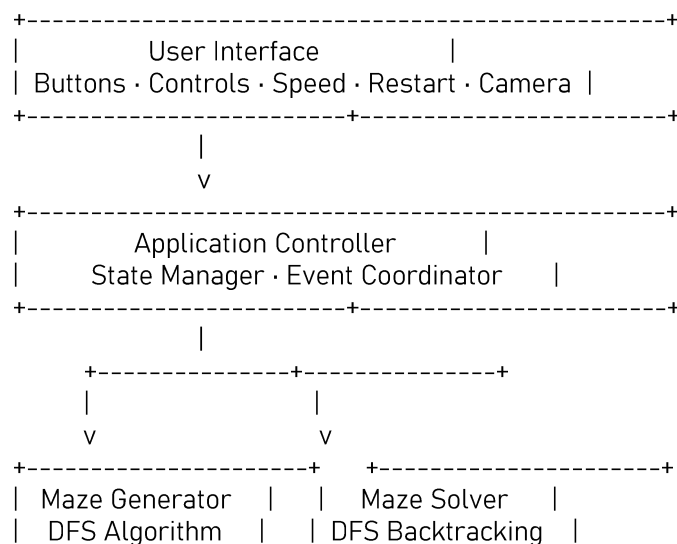
The architecture follows a:

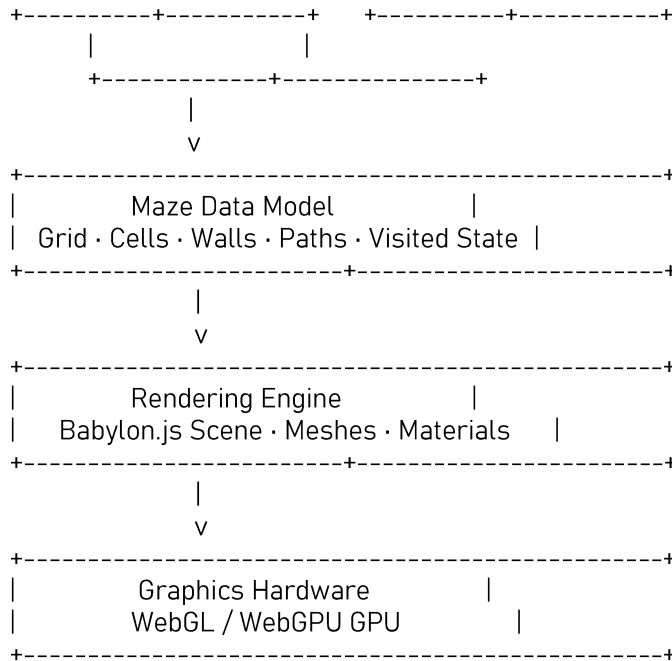
## Modular Event-Driven Layered Design

to separate:

- algorithms,
  - rendering,
  - animation,
  - state management,
  - and UI logic.
- 

## 2. High-Level Architecture





---

## 3. Architectural Pattern

Pattern Used

Layered Modular Architecture

---

### Why This Architecture?

Advantages:

- clean separation of concerns,
  - easier debugging,
  - reusable algorithms,
  - scalable rendering,
  - maintainable codebase,
  - easier testing.
- 

## 4. Core Subsystems

The system is divided into six major subsystems:

| Subsystem        | Responsibility           |
|------------------|--------------------------|
| Core Engine      | App lifecycle and state  |
| Maze System      | Maze data representation |
| Generator System | Maze creation            |
| Solver System    | Maze traversal           |
| Rendering System | Graphics rendering       |
| UI System        | User interaction         |

---

## 5. Core Engine Design

### Responsibilities

The Core Engine manages:

- initialization,
- render loop,
- update cycle,
- scene lifecycle,
- event synchronization.

---

### Core Components

```
Core/  
├── Engine.ts  
├── SceneManager.ts  
├── StateManager.ts  
└── Config.ts
```

---

### Engine Lifecycle

```
Initialize Engine  
↓  
Create Scene  
↓  
Initialize Maze  
↓  
Initialize Renderer  
↓  
Initialize UI
```

↓  
Start Render Loop  
↓  
Update + Render Every Frame

---

## 6. Maze Data Model Design

### Maze Representation

The maze is represented as:

#### Grid Graph Structure

Each cell is a graph node.

Walls represent graph edges.

---

#### Cell Structure

```
class Cell {  
    row: number  
    col: number  
  
    visited: boolean  
  
    northWall: boolean  
    eastWall: boolean  
    southWall: boolean  
    westWall: boolean  
  
    isDeadEnd: boolean  
    isPath: boolean  
}
```

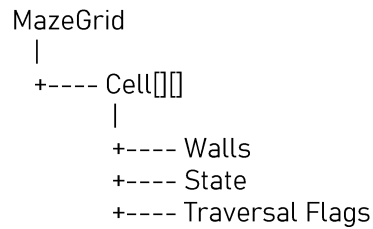
---

#### Grid Structure

```
class MazeGrid {  
    rows: number  
    cols: number  
  
    cells: Cell[][]  
}
```

---

## Relationship Model



## 7. Maze Generator System Design

### Algorithm

#### Randomized DFS Recursive Backtracking

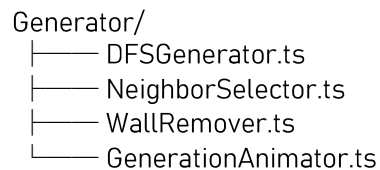
---

### Responsibilities

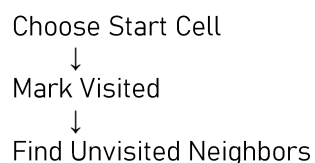
The Generator:

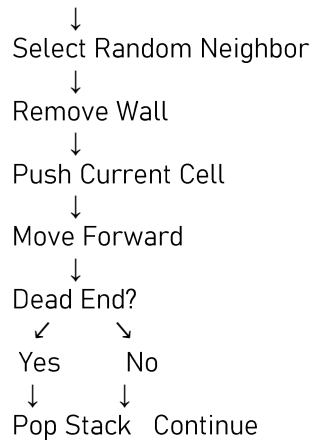
- selects neighbors,
  - removes walls,
  - tracks visited cells,
  - controls generation order,
  - emits animation events.
- 

### Generator Components



### Generator Flow





## 8. Solver System Design

### Algorithm

### DFS Stack-Based Backtracking

---

### Responsibilities

The Solver:

- traverses maze,
- detects dead ends,
- backtracks,
- stores path history,
- emits traversal events.

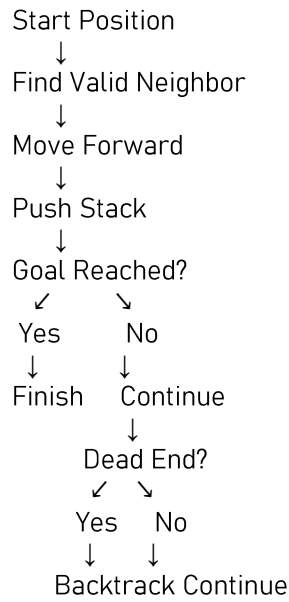
---

### Solver Components

Solver/  
├── DFSSolver.ts  
├── PathTracker.ts  
├── DeadEndManager.ts  
└── SolverAnimator.ts

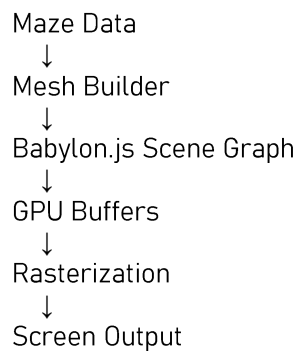
---

### Solver Flow

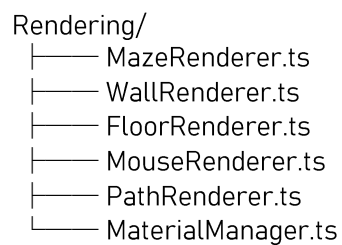


## 9. Rendering System Design

### Rendering Pipeline



### Rendering Components



### Rendering Responsibilities

| Renderer      | Responsibility      |
|---------------|---------------------|
| MazeRenderer  | Entire maze scene   |
| WallRenderer  | Wall mesh creation  |
| FloorRenderer | Ground rendering    |
| MouseRenderer | Mouse visualization |
| PathRenderer  | Path highlighting   |

---

## 10. Mesh Design

### Wall Mesh Strategy

Each wall is represented as:

Thin Box Mesh

Advantages:

- simple collision,
- lightweight rendering,
- easy scaling.

---

### Floor Mesh Strategy

Single large plane mesh.

---

### Mouse Representation

Possible options:

- sphere mesh,
- glowing orb,
- emissive cube.

Recommended:

### Emissive Sphere Mesh

---

## 11. Animation System Design

Animation Model

Event-Driven Incremental Animation

---

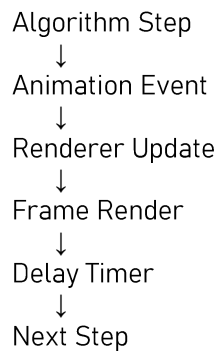
### Animation Responsibilities

The Animator:

- controls speed,
- synchronizes updates,
- schedules state changes,
- emits frame updates.

---

### Animation Flow



---

## 12. Event System Design

Event-Driven Communication

Subsystems communicate through events.

---

### Event Examples

| Event          | Description      |
|----------------|------------------|
| CELL_VISITED   | Generation visit |
| WALL_REMOVED   | Wall destruction |
| SOLVER_MOVED   | Solver movement  |
| DEAD_END_FOUND | Backtracking     |
| PATH_COMPLETED | Solution found   |

---

## Event Flow

Generator  
↓ emits  
EventBus  
↓ notifies  
Renderer

---

# 13. State Management Design

## Global States

```
enum AppState {  
    IDLE,  
    GENERATING,  
    SOLVING,  
    COMPLETED  
}
```

---

## State Transition Diagram

```
graph TD  
    IDLE --> GENERATING  
    GENERATING --> SOLVING  
    SOLVING --> COMPLETED  
    COMPLETED --> RESTART  
    RESTART --> IDLE
```

---

# 14. Camera System Design

## Primary Camera

### ArcRotateCamera

Advantages:

- orbit movement,
- zoom,
- easy debugging,
- top-down visibility.

---

## Camera Modes

| Mode        | Purpose            |
|-------------|--------------------|
| Top Down    | Main visualization |
| Isometric   | Better depth       |
| Follow Mode | Track mouse        |
| FPS Mode    | Bonus feature      |

---

## Camera Architecture

```
CameraManager
|
+----- ArcRotateCamera
+----- UniversalCamera
```

---

# 15. Material System Design

## Material Categories

| Material         | Usage            |
|------------------|------------------|
| WallMaterial     | Maze walls       |
| FloorMaterial    | Ground           |
| MouseMaterial    | Active traversal |
| DeadEndMaterial  | Failed paths     |
| SolutionMaterial | Final path       |

---

## Rendering Style

Recommended:

- low-poly clean style,
  - emissive highlights,
  - high contrast colors.
- 

## 16. UI System Design

### UI Components

UI/  
├── GenerateButton  
├── SolveButton  
├── SpeedSlider  
├── CameraToggle  
└── ResetButton

---

### UI Responsibilities

| Component     | Responsibility        |
|---------------|-----------------------|
| Generate      | Start maze generation |
| Solve         | Start solving         |
| Speed Slider  | Animation speed       |
| Camera Toggle | Switch view           |
| Reset         | Restart simulation    |

---

## 17. Collision Design

Although collision is optional initially, the system supports:

- mouse movement collision,
  - wall boundaries,
  - future FPS traversal.
-

## Collision Strategy

Wall Meshes



Bounding Boxes



Babylon Collision System

---

## 18. Memory Design

### Primary Memory Consumers

| Resource  | Usage          |
|-----------|----------------|
| Cell Grid | Maze structure |
| Meshes    | Wall rendering |
| Stack     | DFS traversal  |
| Materials | GPU resources  |

---

### Optimization Strategies

#### Instanced Meshes

Reuse wall geometry.

#### Shared Materials

Reduce GPU overhead.

#### Single Floor Mesh

Avoid unnecessary geometry.

---

## 19. Error Handling Design

### Potential Errors

| Error               | Solution         |
|---------------------|------------------|
| Invalid grid size   | validation       |
| Empty stack pop     | safety checks    |
| Renderer sync issue | event queue      |
| Infinite traversal  | visited tracking |

---

## 20. Scalability Design

The architecture supports future:

- multiple algorithms,
  - multiplayer traversal,
  - AI pathfinding,
  - procedural environments,
  - larger worlds.
- 

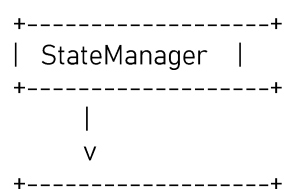
## 21. Future Extensibility

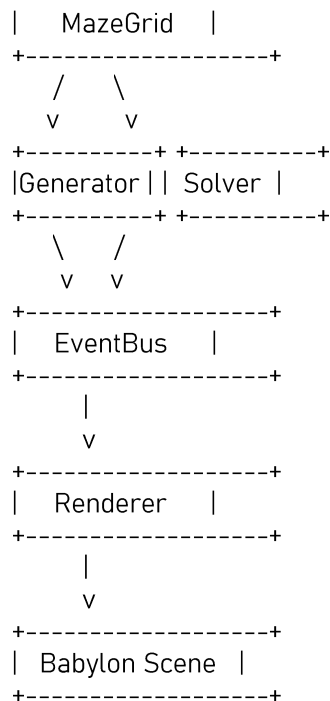
### Planned Extensions

| Feature             | Difficulty |
|---------------------|------------|
| A* Solver           | Medium     |
| BFS Generator       | Medium     |
| FPS Exploration     | Medium     |
| Lighting Effects    | Easy       |
| Shadows             | Easy       |
| WebGPU Optimization | Hard       |

---

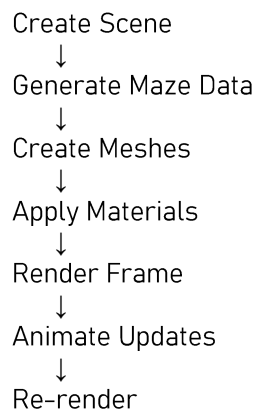
## 22. UML-Style Component Relationships





---

## 23. Rendering Lifecycle



---

## 24. Deployment Design

### Local Development

npm run dev

---

### Production Build

npm run build

---

## Hosting Options

| Platform     | Recommendation |
|--------------|----------------|
| Vercel       | Excellent      |
| Netlify      | Excellent      |
| GitHub Pages | Good           |

---

## 25. Security Considerations

Minimal security concerns since:

- no authentication,
- no backend,
- no database,
- local rendering only.

Main focus:

- dependency safety,
  - package integrity.
- 

## 26. Final System Behavior

The completed system will:

1. initialize a Babylon.js scene,
  2. generate a procedural maze,
  3. dynamically animate wall carving,
  4. visualize DFS traversal,
  5. solve the maze,
  6. backtrack from dead ends,
  7. highlight final path,
  8. provide interactive controls,
  9. render smoothly in real time.
-

# 27. Design Summary

This system design provides:

## Strong Separation of Concerns

Algorithms independent from rendering.

## Scalable Architecture

Supports future extensions.

## Efficient Rendering

Optimized mesh strategy.

## Maintainable Codebase

Modular folder structure.

## Professional Graphics Pipeline

Suitable for:

- computer graphics coursework,
- portfolio presentation,
- algorithm visualization projects.